Tabular Reinforcement Learning for Text Flappy Bird:Monte Carlo Control vs. Sarsa(λ)

Hamza Tbatou

Notebook link: https://github.com/Uchiha-Hamza/3MD3220-Reinforcement-Learning-Individual-Assignment

CentraleSupélec

Abstract. This report compares two tabular reinforcement learning agents on Text Flappy Bird (TFB): constant-step-size first-visit Monte Carlo control and Sarsa(λ) with replacing eligibility traces. The main study uses the compact-state environment `TextFlappyBird-v0`, while the screen-render variant is analyzed to explain why it is not well suited to a purely tabular approach. Under a common training and evaluation protocol across four random seeds, Sarsa(λ) consistently learns faster and converges to much stronger policies. In greedy evaluation on the base configuration, it reaches a mean score of 260.27 ± 222.72, compared with 28.89 ± 38.21 for Monte Carlo. Parameter sweeps show that both methods are sensitive to hyperparameters, but that Sarsa(λ) benefits strongly from eligibility traces, with the best short-budget setting obtained at $\alpha = 0.20$ and $\lambda = 0.70$. Value-function and greedy-policy heatmaps indicate that both agents recover the same broad control strategy, but Sarsa(λ) learns a sharper and more coherent decision boundary. Transfer to unseen level configurations remains limited for both methods, especially when the pipe gap becomes narrower. Overall, online temporal-difference learning with traces is the most effective tabular solution for this task.

Keywords: reinforcement learning · Monte Carlo control · Sarsa(λ) · Text Flappy Bird · tabular RL

1 Introduction and Experimental Setup

The assignment requires a concise comparison of two reinforcement learning agents, together with value-function plots, parameter sweeps, and a discussion of environment design and transfer. The implemented study follows this structure directly.

The main environment is `TextFlappyBird-v0`, using the base configuration

$$(\texttt{height} = 15,\ \texttt{width} = 20,\ \texttt{pipe_gap} = 4).$$

This version provides a compact observation (x, y), where x is the horizontal distance to the next pipe-gap center and y is the vertical offset from that center.

Table 1. Main greedy evaluation on the base configuration.

Agent	Mean return	Std. return	Mean score	Std. score
Monte Carlo	299.15	381.86	28.89	38.21
Sarsa(λ)	2612.92	2226.97	260.27	222.72

Because this state representation is already low-dimensional and task-aligned, it can be discretized directly and handled with a tabular method. By contrast, `TextFlappyBird-screen-v0` returns a full board representation and is therefore inspected only to discuss its limitations for tabular RL.

Both agents act in the same binary action space: *idle* or *flap*. The first agent is constant-α first-visit Monte Carlo control. It stores complete episodes and updates state-action values only after termination. The second agent is Sarsa(λ) with replacing eligibility traces. It performs online temporal-difference updates at every step and propagates credit backward through traces during the episode.

To ensure a fair comparison, both agents are trained under the same protocol on the same environment and are evaluated with greedy policies over the same number of episodes across four random seeds. This setup makes it possible to compare not only final scores, but also convergence speed, parameter sensitivity, and robustness across level configurations.

2 Comparison of the Two Agents

Table 1 reports the main greedy evaluation results on the training configuration. Sarsa(λ) clearly outperforms Monte Carlo on every metric: return, score, and effective survival length. The gap is large enough that it cannot be explained by noise alone.

The learning curves in Fig. 1 tell the same story. Sarsa(λ) improves earlier and keeps improving throughout training, whereas Monte Carlo increases more slowly and plateaus at a much lower level. This difference is expected from the update rules. Monte Carlo waits until the end of an episode before updating, which leads to delayed and high-variance credit assignment. Sarsa(λ), in contrast, updates online and combines bootstrapping with eligibility traces, so useful reward information is propagated much faster through recently visited state-action pairs.

The evaluation boxplots in Fig. 2 confirm that the superiority of Sarsa(λ) is not merely a training-time artifact. Under greedy evaluation, its entire distribution is shifted upward, with much higher medians and much longer upper tails. Monte Carlo still learns a meaningful policy, but it remains substantially weaker.

Overall, the two implemented agents differ in three main ways. First, Sarsa(λ) converges faster. Second, it achieves much higher returns and scores. Third, it is more sensitive to its hyperparameters, because its stronger update mechanism can help or hurt depending on how aggressively it is tuned.

3 Parameter Sensitivity and Learned Representations

The parameter sweeps in Fig. 3 highlight that both algorithms are sensitive to their tuning, but not in the same way. For Monte Carlo, performance depends strongly on the pair (ε, α). Very low exploration leads to poor learning, and the best short-budget setting appears around $\varepsilon = 0.10$ and $\alpha = 0.05$, with mean evaluation score 53.11. Increasing the step size further does not improve performance and tends to make learning more conservative rather than stronger.

For Sarsa(λ), the sweep reveals both higher potential and greater sensitivity. Small step sizes perform poorly, whereas larger values of α produce major gains. The best short-budget configuration is obtained at $\alpha = 0.20$ and $\lambda = 0.70$, with mean evaluation score 176.74. This shows that eligibility traces are not a marginal refinement here: they are one of the central reasons why Sarsa(λ) outperforms Monte Carlo in this task.

The learned value-function and policy heatmaps in Fig. 5 support the quantitative results. Both agents discover the same high-level control principle: flap when the bird is too low relative to the next gap, and stay idle when it is too high. However, the learned representation of Sarsa(λ) is visibly cleaner. Its max-action value function is sharper and more structured, and its greedy policy contains larger contiguous decision regions. Monte Carlo produces a more fragmented boundary, which is consistent with its weaker empirical performance.

4 Environment Design and Generalization

The two available TFB environments differ primarily through their observation spaces. The compact version provides a low-dimensional, highly informative state representation tailored to the task. This makes it ideal for a tabular assignment, but also means that a large part of the representation problem is already solved by the environment design. The screen version is richer and closer to the original game interface, but it is a poor match for a plain Q-table because the number of possible observations becomes too large and too sparse.

This observation also answers the question of whether the same agents can be used on the original Flappy Bird game. Conceptually, yes: Monte Carlo and Sarsa-style learning remain valid RL approaches. Practically, not in the same tabular form. If the original environment is based on pixels or richer continuous dynamics, a direct state-action table becomes infeasible. In such a setting, one would normally replace the table with function approximation, for example using neural networks.

Generalization across level configurations is reported in Table 2 and illustrated in Fig. 4. Transfer remains limited for both agents. The narrow-gap configuration is especially damaging: both methods nearly fail completely. This indicates that the learned policies are tightly coupled to the training geometry. Transfer is better when the environment becomes more forgiving, as in the wide-gap setting, where Sarsa(λ) even exceeds its performance on the base configuration. Changes in level dimensions also hurt performance, although less severely than narrowing the gap.

Table 2. Zero-shot generalization to new level configurations.

Configuration	Monte Carlo score	Sarsa(λ) score
Train config	31.83	261.94
Narrow gap	2.74	1.28
Wide gap	32.36	369.71
Wider level	8.92	52.77
Taller level	15.35	27.50

These results show that although the relative-state representation transfers to some extent, it does not provide strong robustness to distribution shift. The learned policies remain highly specialized to the training regime.

5 Conclusion

This study answers all the points required in the assignment. The two agents differ strongly in convergence behavior, sensitivity, and final performance. Monte Carlo control is simple and learns a non-trivial policy, but it is clearly slower and weaker in this environment. Sarsa(λ) is the better tabular method because it combines online updates, faster credit assignment, and stronger final policies.

The comparison between `TextFlappyBird-v0` and `TextFlappyBird-screen-v0` illustrates a classic trade-off in reinforcement learning. Compact low-dimensional states are easier to solve with tabular methods but less demanding as representation-learning benchmarks. Raw or screen-based observations are more realistic, yet they usually require function approximation rather than a handcrafted Q-table.

Finally, the transfer experiments show that learning on one TFB configuration does not guarantee robust performance on other layouts. Both agents generalize poorly when the environment becomes significantly stricter, especially when the pipe gap narrows. In summary, Sarsa(λ) with replacing traces is the best match for this assignment, while the overall results also make clear where the limits of tabular reinforcement learning begin.

Notebook link.
https://github.com/Uchiha-Hamza/3MD3220-Reinforcement-Learning-Individual-Assignment

References

1. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. 2nd edn. MIT Press, Cambridge (2018)
2. Course assignment material and provided Text Flappy Bird environments, 3MD3220 Reinforcement Learning, 2026.

Figures

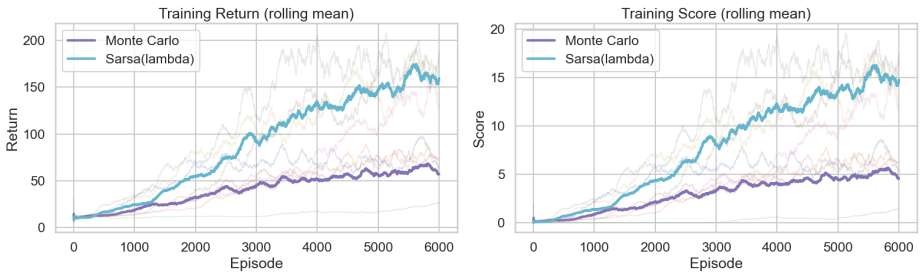

Fig. 1. Rolling training return and score across seeds. Sarsa(λ) learns faster and stabilizes at a much higher level than Monte Carlo.

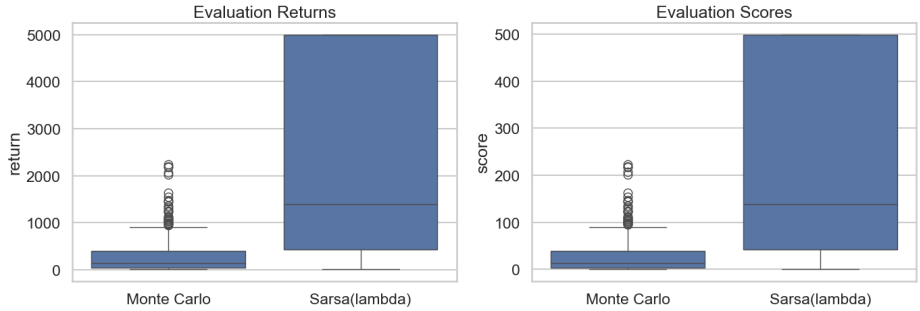

Fig. 2. Distribution of greedy evaluation returns and scores. Sarsa(λ) attains much higher medians and upper tails, confirming that its advantage persists at test time.

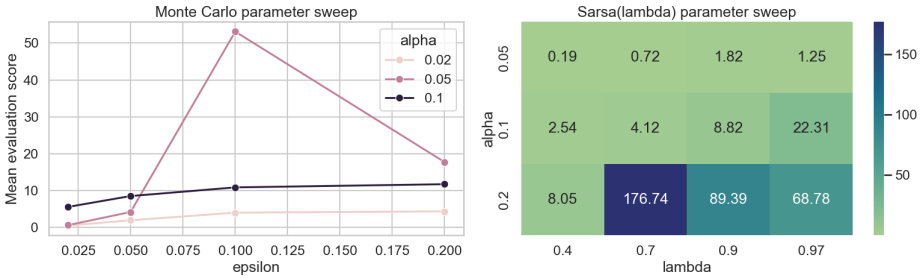

Fig. 3. Parameter sweeps for Monte Carlo and Sarsa(λ). Monte Carlo depends strongly on the (ε, α) pair, while Sarsa(λ) benefits strongly from larger α and intermediate-to-high λ.

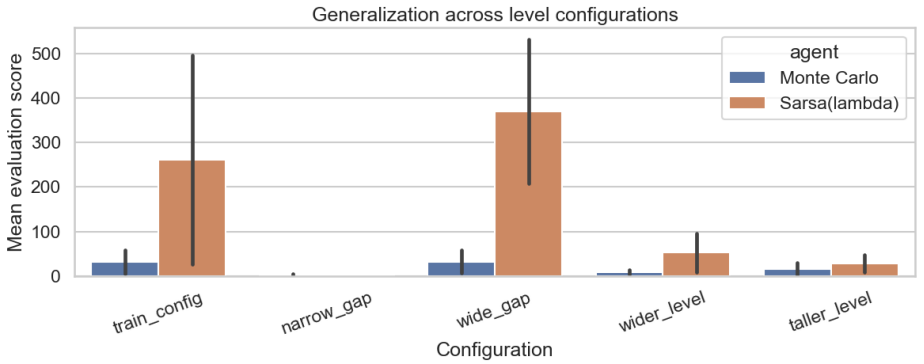

Fig. 4. Zero-shot generalization across level configurations. Transfer is poor when the environment becomes stricter, but better when the layout becomes more forgiving.

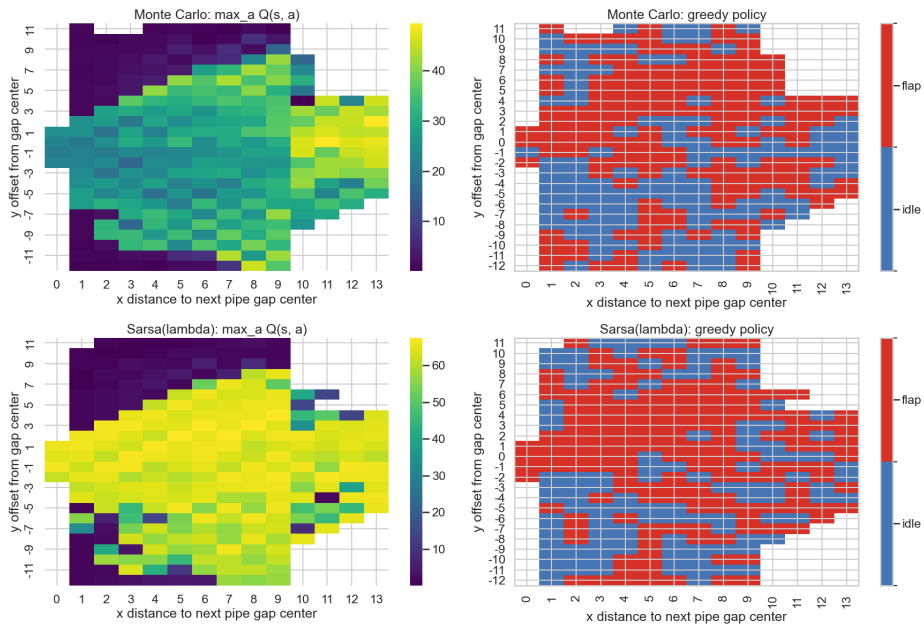

Fig. 5. Learned max-action value maps and greedy policies. Both agents recover the same broad control rule, but Sarsa(λ) learns a clearer decision boundary and higher values in recoverable states.